

Safety and Constant-Time Verification of a Scalar Jasmin NTT for HAETAE

Anonymous Submission

Abstract

The number-theoretic transform (NTT) is a small but security-critical kernel in lattice-based signature implementations. Its low-level code must satisfy two properties before functional correctness is meaningful as an implementation claim: the program must be safe, so that every execution is defined, and it must be constant time, so that branches and memory addresses do not depend on secret polynomial coefficients. This paper reports on the scalar Jasmin implementation of the HAETAE NTT, inverse NTT, and NTT-domain base multiplication at this compiler-side layer. The implementation is stage-specialized so that the Jasmin safety checker can prove every polynomial and twiddle-table access in bounds, and its exported wrappers carry regular and speculative constant-time annotations. We describe how the current Jasmin verification model specializes to this actual source code: typed 256-word polynomial pointers under caller validity preconditions, fixed public loop schedules, public affine address expressions, secret coefficient arithmetic, and entry fences emitted from `#init_ms()`. The checked artifact passes `jasminc -checksafety`, `jasmin-ct`, and `jasmin-ct -speculative`. Together with the Jasmin compiler’s model-relative preservation discipline and byte-for-byte regeneration evidence for `hpoly.s`, these checks give a source-to-assembly safety and branch/address-timing argument for this generated scalar x86-64 code, not an independent proof of arbitrary assembly or all physical side channels. Functional correctness is a separate layer: the companion EasyCrypt development currently proves the forward and inverse NTT wrappers, while base multiplication is covered here only by the safety and timing checks.

1 Introduction

Lattice-based signature schemes spend a large fraction of their running time in polynomial arithmetic. For HAETAE [10], the main transform is a 256-point NTT modulo $q = 64513$. A fast implementation of this transform is not just a mathematical program: it is a low-level in-place routine over machine words, table pointers, loop counters, and ABI-visible entry points. Two classes of mistakes are particularly damaging. First, an out-of-bounds table or polynomial access can invalidate the execution model used by later proofs. Second, a branch or memory address depending on a secret coefficient can leak informa-

tion even if the arithmetic result is correct.

This paper focuses on those two implementation properties for the scalar HAETAE NTT written in Jasmin [1, 2, 14]. The source under `jasmin/` implements three exported routines: `poly_ntt_jazz`, `poly_invntt_jazz`, and `poly_basemul_jazz`. The code is compiled to x86-64 assembly `hpoly.s`. The same source is also suitable for extraction to EasyCrypt for functional verification, but the present paper is about the compiler-side guarantees that make the low-level execution well-defined and side-channel disciplined.

The central claim is deliberately narrow and artifact-based. The scalar Jasmin implementation has been shaped so that the Jasmin toolchain proves the following properties for the exported routines:

- **Safety.** All polynomial and twiddle-table accesses are within the 256-entry regions described by their types, assuming the caller supplies valid and aligned pointer arguments, and the primitive operations used by the program are well-defined in the Jasmin semantics.
- **Regular constant-time.** Branch conditions and memory addresses depend only on public loop counters, public stage constants, and public pointer bases, never on polynomial coefficients.
- **Speculative constant-time.** The exported wrappers establish a speculation boundary with `#init_ms()`, so transient pointer inputs become public address bases while pointed-to coefficients remain secret.

Table 1 records the boundary of the claim up front. This is useful because the paper sits between several verification layers: the NTT mathematics, the Jasmin safety and timing checks, the compiler preservation argument, and the separate EasyCrypt functional proof. The paper uses only the layers named in the evidence column.

The paper does not claim a new NTT algorithm, a vectorized implementation, or a full signature-scheme proof. Its contribution is the implementation-level security argument for the scalar Jasmin kernels, grounded in the actual source and the emitted assembly shape. In particular, it records how a helperized implementation exposes the exact obligations needed by the Jasmin safety, regular constant-time, and speculative constant-time checkers. This is a necessary layer under any functional correctness theorem: a theorem about an extracted model is only an implemen-

Claim area	Evidence used here	Explicit non-claim
Algorithmic target	HAETAE parameters, twiddle-table schedule, scalar Jasmin source	New NTT algorithm or vectorized implementation
Safety	<code>jasminc -checksafety</code> logs and typed pointer preconditions	Validity of arbitrary caller memory
Regular and speculative timing	<code>jasmin-ct</code> , <code>jasmin-ct -speculative</code> , entry <code>lfence</code> shape	All microarchitectural leakage channels
Functional correctness	Companion EasyCrypt forward/inverse NTT theorems	Base-multiplication correctness or full signature security

Table 1: Claim boundaries for the submitted artifact.

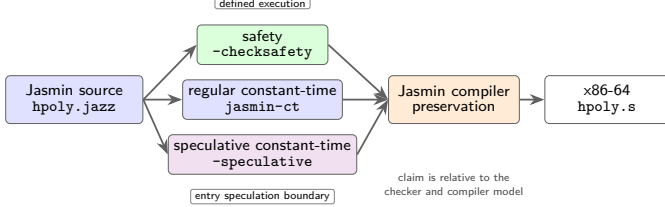


Figure 1: Compiler-side verification composition for the scalar HAETAE NTT artifact. Safety, regular constant-time, and SCT are checked on the same Jasmin source; the compiler-preservation step connects those source-level facts to the generated scalar assembly.

tation theorem if the compiled source is safe and preserves the intended timing discipline.

The paper makes three concrete contributions. First, it gives a code-level description of the scalar HAETAE NTT, inverse NTT, and base-multiplication kernels in the subset of Jasmin that matters to automatic safety and timing checks. Second, it separates the safety, regular constant-time, SCT, compiler preservation, and functional-correctness claims so that each statement has a clear source of evidence. Third, it packages reproducibility evidence for the checked source and the generated scalar assembly. The rest of the paper follows that order: Section 2 describes the implementation target, Sections 3–7 explain the verification models and arguments, Section 8 records the reproduced evidence, and the final sections state the trust boundary and related work.

2 Implementation Target

The artifact consists of five source/support files. `params.jinc` fixes the transform size and modulus; `reduce.jinc` implements Montgomery reduction and field multiplication; `zetas.jinc` contains the forward and inverse twiddle tables; `poly.jinc` contains the stage-specialized kernels; and `hpoly.jazz` contains the exported ABI wrappers. Table 2 summarizes the roles relevant to safety and constant-time verification. The verification arguments below rely on the literal sizes in `params.jinc`: every polynomial pointer has type `u32[HAETAE_N]`, every twiddle table has exactly 256

File	Verification-relevant role
<code>params.jinc</code>	Shared constants <code>HAETAE_Q=64513</code> and <code>HAETAE_N=256</code> .
<code>hpoly.jazz</code>	Exported wrappers, <code>ct/sct</code> annotations, and entry <code>#init_msf()</code> calls.
<code>poly.jinc</code>	Forward NTT, inverse NTT, inverse scaling, and base multiplication loops.
<code>reduce.jinc</code>	Montgomery arithmetic over fixed-width words.
<code>zetas.jinc</code>	Fixed 256-entry forward and inverse twiddle tables.

Table 2: Source files used by the scalar safety and constant-time checks.

entries, `HAETAE_Q = 64513`, and `HAETAE_N = 256`.

2.1 Algorithmic and Functional Context

The implementation target is the negacyclic ring

$$R_q = \mathbb{Z}_q[X]/(X^{256} + 1), \quad q = 64513,$$

with the primitive 512th root $\zeta_0 = 426$ used by the HAETAE reference NTT [10, 13]. At the abstract field level, the forward transform evaluates a polynomial at the odd powers of ζ_0 , arranged in bit-reversed order:

$$\hat{p}[i] = \sum_{j=0}^{255} p[j] \cdot \zeta_0^{(2 \text{brev}_8(i)+1)j}.$$

The inverse transform applies the corresponding inverse matrix and a factor 256^{-1} . The EasyCrypt development used by the companion functional proof follows this full 256-point HAETAE transform, rather than the incomplete 128-point structure used by MLKEM/Kyber.

The Jasmin tables are concrete Montgomery-word realizations of this mathematics [12]. The forward table slot 0 is padding; slots 1–255 contain signed Montgomery representatives of $\zeta_0^{\text{brev}_8(k)}$. The inverse table stores the inverse butterfly constants explicitly in slots 0–254 and reserves slot 255 for the final scale, the signed Montgomery word -29720 , representing $256^{-1}R^2 \bmod q$ for $R = 2^{32}$. The paper’s safety and timing claims therefore apply to the concrete table layout actually read by the scalar source, while the EasyCrypt functional theorem supplies the separate link to the abstract transform for the forward and inverse wrappers.

The public wrappers in `hpoly.jazz` expose three pointer-based procedures. The suffix `p` is used consistently for pointer arguments: `rp` is the result polynomial pointer, while `ap` and `bp` are the two source polynomial pointers for base multiplication. The base-multiplication wrapper marks the source pointers as `const`; the destination pointer is mutable.

Listing 1: Exported wrapper shape in `hpoly.jazz`.

```
#[ct = "rp -> rp"]
#[sct = "{ ptr: transient, val: secret } -> { ptr: public, val:
  secret }"]
export
fn poly_ntt_jazz(reg mut ptr u32[HAETAE_N] rp) -> reg ptr u32[
  HAETAE_N] {
  _ = #init_msf();
  rp = _poly_ntt(rp);
  return rp;
}

#[ct = "rp -> rp"]
#[sct = "{ ptr: transient, val: secret } -> { ptr: public, val:
  secret }"]
export
fn poly_invntt_jazz(reg mut ptr u32[HAETAE_N] rp) -> reg ptr u32[
  HAETAE_N] {
  _ = #init_msf();
  rp = _poly_invntt(rp);
  return rp;
}

#[ct = "rp * ap * bp -> {ap, bp, rp}"]
#[sct = "
{ ptr: transient, val: secret } *
{ ptr: transient, val: secret } *
{ ptr: transient, val: secret } ->
{ ptr: public, val: secret }
"]
export
fn poly_basemul_jazz(reg mut ptr u32[HAETAE_N] rp,
  reg const ptr u32[HAETAE_N] ap bp) -> reg ptr
  u32[HAETAE_N] {
  _ = #init_msf();
  rp = _poly_basemul(rp, ap, bp);
  return rp;
}
```

The wrapper code is intentionally thin. It does not perform arithmetic itself; it establishes the security boundary and then calls an internal kernel. This separation is useful for verification. The exported function is where the caller-provided pointer enters the checked code, so this is where the `sct` contract and `#init_msf()` appear. The internal kernels then run under the simpler invariant that pointer bases are public address bases and coefficient values are secret data.

2.2 Jasmin Implementation Discipline

Jasmin is used here as a structured assembly language rather than as a high-level source language. The code makes the machine-level objects that matter to verification explicit: register variables are declared with `reg`, caller-provided arrays are typed as `ptr u32[HAETAE_N]`, and mutability is visible at the function boundary. A declaration such as

```
reg mut ptr u32[HAETAE_N] rp
```

is therefore more than documentation. It gives the checker a typed 256-word shape for indexed accesses through `rp`, tells the compiler that the pointer is passed in a register, and records that the routine may write through the pointer. It does not by itself allocate or validate memory: the initial state must still provide a valid, aligned region satisfying the pointer precondition. The source pointers of `poly_basemul_jazz` are instead `reg const ptr u32[HAETAE_N]`, which matches the implementation discipline that `ap` and `bp` are read-only inputs.

The Jasmin documentation explains the source structure used by this artifact: programs are assembled from **required** files, parameters, global variables, type aliases, and functions. This is exactly the organization in Table 2. The constants in `params.jinc` are parameters, so they may appear in array sizes such as `u32[HAETAE_N]` and are resolved early by the compiler. The twiddle tables in `zetas.jinc` are global arrays stored in immutable code memory. The array documentation also explains why the implementation introduces `reg ptr u32[256] zetasp` before reading `jzetas`: a `reg ptr` is the Jasmin idiom for holding the address of a global array or caller-provided array in a register.

Array indexing has the usual zero-based meaning, but the index is implicitly scaled by the declared element size. Thus `rp[idx]` denotes the `u32` cell at byte offset `4*idx` from the public base address `rp`. This matters to both safety and timing. Safety needs `idx < 256`, not merely a byte offset bound, while the timing checker sees that the emitted address is a public base plus a public scaled index. The code avoids register arrays with run-time indices, which the manual warns must be statically known after inlining and unrolling. Instead, the polynomial and table objects are memory arrays, for which run-time public indices are the intended use case. The 2026 Jasmin material also records newer register spilling support, including auto-spill modes, but this artifact does not rely on those features: it is compiled with the ordinary command line recorded in Section 9.

The scalar-type documentation is also relevant. Coefficients are machine words (`u32`), whose arithmetic is the usual wrapping word arithmetic. Loop and index variables are word-sized integers (`ui64`), which behave like bounded mathematical integers in the source semantics and are compiled to machine words later. Operations producing such values carry range assertions: if a `ui64` computation overflowed its source-level range, execution would be stuck and the safety checker would have to reject the program. In this implementation, the small stage counters and offsets make those range assertions straightforward.

The NTT kernels avoid features that would make the low-level reasoning unnecessarily indirect. They allocate no source-level arrays, perform no raw byte-addressed loads such as `[p + e]`, use no recursion, and call only small arithmetic helpers whose effects are visible in the same Jasmin source unit. The observable memory operations are therefore precisely the typed polynomial accesses, the typed twiddle-table accesses, and the final stores back to the result polynomial. This is why the proof obligations can be read from the source: every address is a base pointer plus a public word index, and every non-address data dependency is coefficient arithmetic in registers.

The implementation also separates the public NTT schedule from the secret data flow. The usual in-place butterfly for a stage of length ℓ reads two cells r_i and $r_{i+\ell}$, computes a Montgomery product $t = \zeta r_{i+\ell}$, and writes $r_i + t$ and $r_i - t$ back to the same two publicly deter-

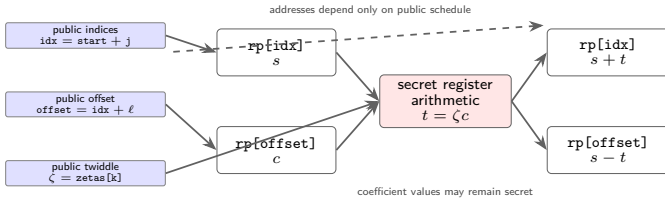


Figure 2: One butterfly instance as seen by the safety and constant-time checks. The public loop schedule determines the two addressed cells and the twiddle-table entry, while coefficient-dependent values stay in register arithmetic and stored data.

mined positions. In the Jasmin code the schedule variables `block`, `j`, `start`, `idx`, and `offset` are public loop counters or expressions built from them. The variables `s`, `coeff`, and `t` carry coefficient values and are allowed to be secret. The code is therefore arranged so that the checker sees a syntactic separation between the public schedule and the secret arithmetic payload.

Stage specialization is the main implementation device used to make this separation automatic-checker friendly. A compact C-style NTT loop would carry a mutable `len` variable and would require the checker to relate `len`, `start`, `j`, and table offsets across nested loops. The Jasmin artifact instead expands the eight stages into eight helpers. Each helper has literal loop bounds and literal offsets, so the checker sees ordinary integer facts such as `block < 4`, `j < 32`, and `offset = start + j + 32`. This does not change the NTT algorithm; it changes the presentation of the program so that the safety and timing invariants are local to each helper.

2.3 Montgomery Arithmetic Layer

The arithmetic layer in `reduce.jinc` follows the C reference convention for HAETAE. The constant `QINV` is $q^{-1} \bmod 2^{32}$, with $q = 64513$. The reduction procedure first truncates the input to 32 bits, multiplies by `QINV` with wrapping 32-bit semantics, sign-extends that word, subtracts the corresponding multiple of q , and performs an arithmetic right shift by 32 bits. The Jasmin code makes each width change explicit:

Listing 2: Montgomery arithmetic in `reduce.jinc`.

```
inline fn __montgomery_reduce(reg u64 a) -> reg u32 {
  reg u32 t32;
  reg u64 t64;

  t32 = (32u)a;
  t32 *= QINV;
  t64 = (64s)t32;
  t64 *= HAETAE_Q;
  a -= t64;
  a >>= 32;
  t32 = (32u)a;
  return t32;
}

inline fn __fqmul(reg u32 a b) -> reg u32 {
  reg u64 ta tb t;
  reg u32 r;

  ta = (64s)a;
  tb = (64s)b;
  t = ta * tb;
  r = __montgomery_reduce(t);
  return r;
}
```

This layer is important for the constant-time argument. Coefficients and products are secret, but the operations in Listing 2 are arithmetic operations on registers. They do not choose an instruction path and they do not compute a memory address. The Jasmin x86 checker therefore needs to accept the selected arithmetic instructions under its constant-time instruction model, while still forbidding the same secret values from reaching branches or addresses.

2.4 Forward Transform

The forward transform is written as eight fixed helpers. The wrapper `_poly_ntt` calls them in descending stage-length order:

128, 64, 32, 16, 8, 4, 2, 1.

Each helper fixes a pointer `zetasp` to the forward table `jzetas`, iterates over a public block counter, loads a public twiddle-table entry, and then performs a public inner loop over the butterflies in that block. The first helper illustrates the pattern:

Listing 3: First stage-specialized forward NTT helper.

```
fn _poly_ntt_stage_128(reg mut ptr u32[HAETAEN] rp)
-> reg ptr u32[HAETAEN] {
  reg ptr u32[256] zetas;
  reg ui64 block j start idx offset;
  reg u32 zeta t s coeff;

  zetas = jzetas;
  block = 0;
  while (block < 1) {
    zeta = zetas[1 + block];
    start = block;
    start <<= 8;
    j = 0;
    while (j < 128) {
      idx = j;
      idx += start;
      s = rp[idx];
      offset = idx;
      offset += 128;
      coeff = rp[offset];
      t = __fqmul(zeta, coeff);
      coeff = s;
      coeff -= t;
      rp[offset] = coeff;
      s += t;
      rp[idx] = s;
      j += 1;
    }
    block += 1;
  }
  return rp;
}
```

Two details make this code friendly to the automatic checker. First, all indices are unsigned 64-bit loop variables or expressions built from them. Second, the stage length appears as a literal in the helper: the checker sees `block < 1`, `j < 128`, and `offset = idx + 128` in the first helper, rather than having to infer a relation between a mutable `len` variable and several nested loops. The later helpers instantiate the same proof shape with `block < 2`, `j < 64`; `block < 4`, `j < 32`; and so on. This is not an algorithmic change relative to the direct Cooley–Tukey schedule; it is an implementation choice that exposes the public stage constants to `jasminc`.

2.5 Inverse Transform and Final Scale

The inverse transform mirrors the same implementation strategy in the opposite stage order. It starts at length 1 and ends at length 128, then applies a separate final scaling loop. The inverse helpers read from `jzetas_inv`; their table bases are literal offsets

0, 128, 192, 224, 240, 248, 252, 254.

The final scale reads `jzetas_inv[255]` and multiplies every coefficient by that fixed factor. Treating the final scale as a separate helper keeps both the safety and timing arguments simple: its only index is the public loop counter `j`, with guard `j < HAETAEN`.

The inverse butterfly also has no data-dependent control. It reads `rp[idx]` and `rp[offset]`, computes the sum and Montgomery-scaled difference, and stores both results back. The arithmetic values are secret, but the pair of touched locations is determined solely by the public stage length, public block counter, and public inner-loop counter.

2.6 Base Multiplication

The base-multiplication kernel is the simplest example of the memory discipline: it performs a public 256-iteration loop, reads one word from each source polynomial, computes a Montgomery product, and writes one word to the result polynomial.

Listing 4: Base multiplication in `poly.jinc`.

```
fn _poly_base_mul(reg mut ptr u32[HAETAEN] rp,
  reg const ptr u32[HAETAEN] ap bp) -> reg ptr u32[
  HAETAEN] {
  reg ui64 i;
  reg u32 a b r;

  i = 0;
  while (i < HAETAEN) {
    a = ap[i];
    b = bp[i];
    r = __fqmul(a, b);
    rp[i] = r;
    i += 1;
  }
  return rp;
}
```

3 Jasmin Verification Model

Jasmin was designed to give programmers assembly-level control while keeping source-level verification tractable [1, 2]. The 2026 framework material is the current baseline used here: it presents Jasmin as a predictable structured-assembly language, stresses source-level safety and constant-time checks, and states the information-flow rule that branch conditions and memory indices must be public [14]. The older papers and slides give the theoretical and instrumental background; the 2026 slides fix the terminology and tool expectations used in this paper [11, 8, 6].

The two core principles are unchanged. Safety is a definedness property: every memory access must be valid, every array index must be in bounds, and primitive operations must be used within their semantic domains. constant-time security is a trace property: two executions that agree on public inputs must produce the same sequence of branches and memory read/write operations, even if their secret state differs.

3.1 Theoretical Foundation

The theoretical basis used in this paper is layered. The original Jasmin work gives the language a machine-checked semantics, provides source analyses for safety and constant-time, and proves that the compiler preserves source behavior and constant-time under its model [1]. The Last Mile work extends this view into a methodology for developing optimized cryptographic implementations: write code in Jasmin, prove or check source-level functional and implementation-security properties, then rely on the predictable verified compiler to connect those properties to assembly [2]. The 2021 and 2023 Jasmin workbench material presents the same separation in tool form: safety

Source family	Role in this paper
Jasmin CCS and Last Mile papers	Formal semantics, checked source properties, predictable verified compilation, and source-to-assembly constant-time preservation.
2021–2026 Jasmin slides	Current tool interpretation: old versus modular safety checking, explicit CT annotations, EasyCrypt extraction, and the 2026 framework terminology.
Spectre/SCT material	Three-level <code>secret/public/transient</code> typing, MSF discipline, <code>#init_msf()</code> lowering, and the Spectre-v1 scope of SCT [9, 6].
Kyber implementation-correctness paper	NTT proof precedent separating field-level functional correctness from implementation safety and timing claims.
2025 compiler-security paper	Boundary marker for cryptographic-security preservation; complementary to this artifact-level safety/CT/SCT result.

Table 3: How the non-main dissertation PDFs support the theory used here.

checking, EasyCrypt extraction for functional reasoning, and explicit leakage traces for implementation security [11, 8]. The 2024–2026 course material is the current standard adopted here: it distinguishes functional correctness (including safety), implementation security, and cryptographic-security reductions; it also records the current safety-checking workflow and the source information-flow rule that branch conditions and memory indices must be public [6, 14].

This layered reading also fixes what each local claim can and cannot mean. The Kyber implementation-correctness paper shows how Jasmin and EasyCrypt have been used for lattice NTTs: finite-field representation, twiddle-table schedules, bit-reversal, and optimized loop orders are handled by intermediate programs and relational proofs, while implementation-security checks remain a separate layer [3]. That precedent supports the division used here: this paper proves safety and timing properties of the HAETAE scalar NTT source, while functional NTT correctness belongs to the companion EasyCrypt development. Likewise, the recent compiler-security paper formalizes a stronger but different question, namely preservation of cryptographic security experiments for the Jasmin compiler front-end using interaction trees and RHL [4]. It is cited to position the work, not as an unstated proof of HAETAE signature security, compiler back-end security, or side-channel security for this artifact.

3.2 Threat and Leakage Model Used Here

The implementation-level adversary considered here observes the leakage trace modeled by the Jasmin x86 checkers, not arbitrary physical side channels. The public inputs are the selected routine, pointer bases, loop bounds, table layout, stage constants, and fixed twiddle-table locations. The secret state is the polynomial coefficient data and every arithmetic value derived from those coefficients. The regular constant-time check treats branch decisions, loop

guards, memory addresses, array indices, and non-constant-time instruction operands as observable. The intended invariant is therefore precise: changing the coefficients may change register values and stored coefficient values, but must not change the executed control-flow path or the sequence of memory locations touched by the program.

The speculative checker extends this model in the Jasmin speculative constant-time/Spectre-v1-style sense used by the tool. Exported wrappers receive transient pointer inputs and fence the boundary before calling the internal kernels; inside the kernels, pointer bases and indices are public while loaded coefficient values remain secret. This is a model-relative statement. It does not claim coverage of every microarchitectural channel, such as power, electromagnetic leakage, frequency effects, replacement-policy state, or prefetch behavior not represented by the checker. It also does not remove the ordinary ABI precondition that the caller passes valid, aligned polynomial regions.

3.3 Principle of the Safety Check

The safety checker is an abstract interpreter for the Jasmin source semantics. It does not prove that the NTT computes the right polynomial transform; it proves that the program never steps outside the semantic domain assumed by such a functional proof. For this artifact, the important safety preconditions come from the exported types: a caller must pass valid 256-word `u32` regions for the polynomial pointers, and the global twiddle tables must have the declared 256-entry layout. Under those preconditions, the checker generates and discharges local obligations for each command.

The manual description of `-checksafety` is useful for interpreting the output. The flag checks the safety of exported functions rather than simply emitting assembly, and the reported result is conditional on a safety precondition for the initial state. The documented notion of safety includes termination, in-bounds array accesses, valid and properly aligned allocated memory, and valid arguments to arithmetic operations. Thus `No Safety Violation` for `poly_ntt_jazz` should be read as follows: assuming the ABI supplies a valid aligned polynomial region for `rp`, all calls reached from that export terminate without undefined Jasmin behavior. The same reading applies to `poly_invntt_jazz` and `poly_basemul_jazz`, with the latter adding valid aligned input regions for `ap` and `bp`.

The 2026 Jasmin slides distinguish this command from a newer modular safety workflow based on contracts, source assertions, automatic discharge, and EasyCrypt proofs for remaining assertions [14]. This paper intentionally uses `jasminc -checksafety`: the scalar NTT has no symbolic heap structure, no source stack arrays, and no data-dependent loop bounds, and the stage-specialized helpers expose exactly the small arithmetic facts that the old static checker can discharge. The successful run is therefore evidence about this concrete artifact, not a claim that the old checker is the preferred method for every modern Jasmin

program.

The most visible obligations are array bounds and memory validity. For an access `rp[idx]`, the checker must know that `rp` denotes a valid typed region and that the current abstract state entails $0 \leq \text{idx} < 256$. For a store `rp[offset] = coeff`, it must prove the same bound for `offset` and must also know that the pointer is mutable. The checker also tracks definedness of word-level operations: casts, signed and unsigned interpretations, shifts, multiplications, and calls to helpers must all be used in the way the Jasmin semantics permits. Thus a successful safety run should be read as a conditional statement: if the ABI caller supplies valid typed regions, every source execution of the checked routine is defined and every memory access stays inside its declared object.

This is the right safety principle for the scalar NTT because the code’s memory shape is simple. The checker is not required to reason about alias-free heap allocation, symbolic byte offsets, or data-dependent loop termination. It only has to propagate public counter ranges through the stage-specialized loops and through the table-index expressions. The resulting proof obligations are small integer facts, and their connection to the source code is visible in each helper’s literal loop bounds.

3.4 Principle of Constant-Time Verification

The regular constant-time checker is an information-flow analysis for the leakage trace of the program. The programmer supplies a security signature for each exported function. The checker then propagates labels through expressions and commands. Values marked secret may be computed, combined, and stored; secrecy is not an error by itself. The error condition is a secret dependency reaching something that the leakage model treats as observable: a branch condition, a loop guard, a memory address, an array index, or an instruction operand that the selected architecture model does not classify as constant time.

The Jasmin documentation and the recent slides distinguish two constant-time methodologies: the largely automatic type-system check used by `jasmin-ct`, and more expressive proof-oriented methods for broader probabilistic notions. This artifact uses the automatic checker. Its documented rules are exactly the ones needed by the scalar NTT. Constants are public; the label of an arithmetic expression is the maximum of the labels of its operands; array indices and raw memory addresses must be public; `if` and `while` conditions must be public; and exported functions must carry explicit `ct` signatures unless inference is requested for development. The code uses no `#declassify` escape hatch, so the proof does not rely on an unverified claim that a secret-derived value has become public [8, 14].

This gives a precise meaning to the common implementation rule “do not branch or index on secrets.” In the NTT code, polynomial coefficients are secret and twiddle values are public table constants. A coefficient may

flow through `__fqmul`, additions, subtractions, casts, and stores. It may not flow into `j < 128`, into `1 + block`, into `idx`, or into `offset`. If it did, the checker would reject the program because two runs with different coefficients could take different control paths or touch different addresses. Conversely, the public loop counters and fixed stage constants are allowed to determine both branches and addresses.

The speculative checker strengthens this principle at exported boundaries. The documentation and the 2024–2026 slides present this as selective speculative load hardening for Spectre v1-style transient execution [6, 14]. Its key idea is that cryptographic code already knows which values are public and which are secret, so the hardening discipline can focus on public loads and pointer values whose transient behavior must be controlled. The SCT type system separates the security of a pointer value from the security of the memory value reached through that pointer and distinguishes normal execution from transient execution. A transient pointer is architecturally public but is not treated as freely public for speculative addressing until the misspeculation-flag state has been initialized. The manual states that exported-function arguments are treated at least as `transient`, because an export cannot receive an MSF argument and the compiler defensively assumes that execution may already be misspeculating at entry. The wrappers in Listing 1 therefore call `#init_msf()` before entering the arithmetic kernels. In the documented x86 lowering, this operation emits an `lfence` and initializes the MSF to the exact state. After that boundary, the checker may type the pointer bases as public while continuing to type the pointed-to polynomial coefficients as secret. The resulting claim is model-relative: it is evidence in the Jasmin SCT checker’s Spectre-v1-style leakage model for this source and call graph, not a measurement claim and not a proof against every speculative or microarchitectural channel.

The discussion here is grounded in portable artifact components rather than in machine-specific paths. The implementation is under `jasmin/`; the raw reproduction logs are under `paper/logs/`; and the Jasmin documentation pages used to interpret the tools are identified by their source-tree-relative names: `source/tools/safety_checker.md`, `source/tools/ct.md`, `source/tools/sct.md`, `source/language/semantics/arrays.md`, `source/language/semantics/scalar_types.md`, and `source/compiler/passes/index.md`. The commands below were reproduced with repository revision `f0e1399` and Jasmin compiler version `2026.03.0`. The checker components in Table 4 are likewise named relative to a Jasmin source checkout or artifact container. This matters because the paper does not ask the reader to accept a folklore description of Jasmin verification: the manual describes the intended tool interfaces, and the compiler source contains the security annotation language, export-function annotation checks, and speculative pointer/value type discipline.

The annotation parser explains the notation in Listing 1.

Jasmin component	Role in this paper
<code>compiler/safetylib</code>	Abstract-interpretation infrastructure used by <code>jasminc -checksafety</code> .
<code>compiler/src/securityAnnotations.ml</code>	Parser for <code>ct/sct</code> signatures, including named CT levels, arrows, products, <code>transient</code> , and pointer/value SCT types.
<code>compiler/src/ct_checker_forward.ml</code>	Regular constant-time checker; exported functions must be fully annotated unless inference is requested, and result labels must be justified by input labels.
<code>compiler/src/sct_checker_forward.ml</code>	Speculative checker; models direct and indirect security types, enforces export-boundary constraints, and handles the MSF discipline used by <code>#init_msfc()</code> .
<code>compiler/entry/jasmin-ct.ml</code>	Command-line entry point for <code>jasmin-ct</code> and <code>jasmin-ct -speculative</code> .

Table 4: Jasmin checker components used to interpret the artifact.

For regular constant-time contracts, an input type can be public, secret, unspecified, or a named level such as `rp`, `ap`, or `bp`. An output type may mention a set of such input levels. Thus `rp * ap * bp -> {ap, bp, rp}` is not a mathematical product over polynomials; it is a security signature saying that the returned value may depend on the security levels assigned to the three input arguments. For speculative contracts, `securityAnnotations.ml` defines indirect types of the form `{ptr :  $\ell_p$ , val :  $\ell_v$ }`. It also defines `transient` as the level whose normal component is public and whose speculative component is secret. That is precisely the entry type used by the exported wrappers.

The present implementation uses a small and regular subset of the Jasmin memory model. The source has no raw memory expression of the form `[p + e]` and no source-level stack array. Its observable source memory operations are typed array and pointer operations:

`rp[idx], rp[offset], ap[i], bp[i],`

plus reads from the fixed global tables `jzetas` and `jzetas_inv`. Thus the byte-block memory obligations from the general Jasmin model specialize to word-array obligations: each caller-provided polynomial pointer must designate a valid 256-word u32 region, and every computed index must lie in `[0, 256)`.

The constant-time checks are relative to the Jasmin x86 leakage model. Branches and memory addresses are observable. Arithmetic instructions that the model classifies as constant time do not add secret-dependent leakage obligations. This distinction is important for this code. Montgomery multiplication lowers to arithmetic such as `imul` and `sar` on coefficient-derived registers. The checker accepts this arithmetic under the selected x86 model, but it would reject any use of the same coefficient-derived values in branch conditions, loop bounds, memory indices, or operations marked non-constant-time by the architecture model.

4 Why the Checks Are Convincing

The verification argument is not a collection of informal coding conventions. It is a composition of three explicit checks over the actual Jasmin source. Each check corresponds to a semantic obligation, and each obligation rules out a different way in which an implementation theorem could be misleading.

The intended theorem can be stated as a conditional non-interference claim. Let two calls to one exported routine satisfy the same public ABI inputs, the same typed pointer-validity preconditions, and possibly different polynomial coefficients in the pointed-to arrays. If both calls are run through the Jasmin semantics and then compiled by the Jasmin compiler, they follow the same branch schedule and touch the same sequence of memory addresses. They may write different coefficient values, because those values are secret outputs, but the observable control/address trace is independent of those values. The safety check supplies the defined-execution premise of this statement; the regular constant-time check supplies the trace-noninterference premise; the speculative constant-time check supplies the corresponding entry-boundary premise for transient execution; and the compiler argument connects these source properties to the generated x86-64 code.

The pieces of this composition should not be conflated. The safety statement comes from `jasminc -checksafety` and is conditional on valid initial memory. The branch/address timing statement comes from `jasmin-ct` and the Jasmin compiler’s constant-time preservation discipline as presented in the current framework material [14]. The speculative statement comes from the separate SCT checker and its Spectre-v1-style model. Recent work on cryptographic-security preservation for the Jasmin compiler front-end is related and important, but it proves a different theorem for KEM IND-CCA security and does not by itself instantiate a full HAETAE signature, back-end, or side-channel theorem for this artifact [4].

This chain is persuasive only if the checker obligations align with the actual program structure. In this artifact they do. Table 5 lists the verification obligations and the concrete source facts that discharge them. None of the obligations relies on a hidden algebraic property of the NTT or on a probabilistic assumption about coefficient values. They are structural facts about loops, indices, pointer types, and the placement of arithmetic operations.

Safety establishes defined execution. The command `jasminc -checksafety` invokes the safety analysis under `compiler/safetylib`. In particular, `safetyInterpreter.ml` builds safety conditions for array accesses, sub-array accesses, memory validity, word-operation side conditions, and return conditions, then reports `No Safety Violation` only when the abstract state entails those conditions. This matters because constant-time is normally a property of traces of executions; if the program could go out of bounds or execute an invalid

Obligation	Concrete reason it holds in the artifact
Polynomial accesses are in bounds	Every exported polynomial pointer has type <code>ptr u32[256]</code> , and all indices are public counters bounded by literal stage sizes.
Twiddle accesses are in bounds	Forward stages use table entries 1–255; inverse stages use entries 0–254; the final inverse scale uses entry 255.
Branches are public	All loop guards compare public counters with constants such as 128, 64, or <code>HAETAEN</code> .
Addresses are public	Address expressions contain pointer bases, public counters, and literal stage offsets, never coefficient-derived words.
Secret values remain allowed where needed	Coefficients and Montgomery products flow through register arithmetic and stores, which is exactly the intended secret-data path.
Exported speculative boundary is explicit	Each exported wrapper calls <code>#init_msf()</code> before entering the kernel, and the <code>sct</code> signature separates pointer security from pointed-to value security.

Table 5: Checker obligations matched to source-level implementation facts.

primitive operation, the trace argument would be about the wrong semantics. Safety is therefore the first layer: it says that every trace considered by the timing checker corresponds to a defined low-level execution under the typed pointer preconditions.

For this artifact the generated safety obligations are simple enough to audit against the code. The only caller-provided memory regions are the polynomial pointers `rp`, `ap`, and `bp`; the only global arrays are the 256-entry twiddle tables. The stage-specialized code makes all array indices public affine expressions of bounded counters. Consequently, the safety checker is not being asked to discover a cryptographic invariant about secret data. It is checking arithmetic facts such as $j < 128 \Rightarrow j + 128 < 256$ in the first forward stage, and $i < 256$ for `ap[i]`, `bp[i]`, and `rp[i]` in base multiplication.

Regular constant-time is enforced as information flow to the leakage trace. The regular checker in `ct_checker_forward.ml` is a type system for the parts of the program that may affect the timing trace. Its expression checker has a public mode: when an expression is used as an array index, memory address, `if` condition, `while` guard, or bound, the checker calls the expression typing function with the requirement that the result be public. For operations classified as not constant time by the architecture model, the checker similarly requires public operands. Exported functions are required to have complete security annotations unless inference is explicitly requested, and result labels may mention only levels that appear in the input signature.

This gives a precise interpretation to the successful `jasmin-ct` run. It does not merely say that a reviewer failed to notice a secret branch. It says that the checker traversed the program and proved that every expression contributing to the leakage trace is public in the specified security environment. In the HAETAEN NTT code, the

secret values are exactly the loaded polynomial coefficients and the arithmetic values derived from them. Those values flow through `+`, `-`, casts, Montgomery multiplication, and stores. They do not flow to loop guards, branch conditions, or address expressions. The checker enforces precisely that separation.

Speculative constant-time validates the exported boundary. The speculative checker in `sct_checker_forward.ml` strengthens the regular property by distinguishing normal and speculative security levels and by splitting pointer security from pointed-to-value security. The local annotation parser defines `transient` as normal-public but speculative-secret. Thus a wrapper argument of type

`{ptr : transient, val : secret}`

means that the pointer is architecturally public but is treated cautiously under transient execution until the microarchitectural speculation state is initialized. The checker also tracks MSF state: `#init_msf()` is a special operation in the SCT checker, and in the x86 lowering used here it corresponds to an `lfence` at the exported entry point.

The successful `jasmin-ct -speculative` run therefore rules out a specific false proof pattern. The proof is not allowed to assume that an ABI-provided pointer is safely public for speculative addressing before the entry hardening point. After `#init_msf()`, the bodies of `_poly_ntt`, `_poly_invntt`, and `_poly_basemul` use public pointer bases and public counters for addresses, while the values loaded from the arrays remain secret. This is exactly the policy required for exported cryptographic routines.

The compiler argument connects source checks to assembly. The final step is preservation through compilation. The Jasmin compiler is designed for predictable low-level transformations and is accompanied by the compiler-correctness development described in the Jasmin paper and subsequent framework material [1, 2, 14]. The safety and timing argument here uses that discipline: the source-level fixed schedule may be lowered to branches and indexed memory operands, but the compiler is not allowed to introduce table lookups indexed by secret values or branches on secret-dependent ranges. This design choice is what makes the source-level checks meaningful evidence for the generated `hpoly.s`. The claim is not that arbitrary optimizing compilation would preserve constant-time; the claim is that this source is checked by the Jasmin toolchain and compiled by the Jasmin compiler under its preservation model.

The composition used in this paper is therefore the following artifact theorem schema. First, `jasminc-checksafety` proves that the three exported source routines have defined executions under their typed pointer preconditions. Second, `jasmin-ct` proves that their branch and memory-address leakage trace is independent of coefficient values. Third, `jasmin-ct -speculative` proves the

same source against the Jasmin SCT checker’s transient-pointer model. Fourth, `jasminc` compiles that source to scalar x86-64 assembly, and the regenerated assembly is byte-for-byte identical to the submitted `hpoly.s`. The conclusion is conditional on exactly these premises: the submitted assembly is the compiler output of the checked source, and its safety and timing claim is the one preserved by the Jasmin model. It is not a separate binary-analysis theorem, and it does not extend to hand-edited assembly.

Why this is persuasive for this implementation.

The implementation is intentionally shaped so that the checked proof obligations coincide with the informal cryptographic discipline. There are no data-dependent loops, no data-dependent conditionals, no raw source-level pointer arithmetic, and no table lookup indexed by a coefficient. The only variables that influence addresses are public counters and literal stage constants; the only variables that carry secrets are coefficients and coefficient-derived arithmetic temporaries. The checker outputs are therefore not a black-box certificate for an opaque program. They confirm an auditably simple invariant: public schedule and public addresses, secret arithmetic values.

5 Safety Argument

The safety proof rests on the fact that all memory locations are selected by public counters with literal bounds. For a forward stage of length ℓ , the helper has the schematic form

$$\text{block} < \frac{256}{2\ell}, \quad j < \ell, \quad \text{start} = \text{block} \cdot 2\ell,$$

with

$$\text{idx} = \text{start} + j, \quad \text{offset} = \text{idx} + \ell.$$

The checker therefore needs only local arithmetic to prove

$$0 \leq \text{idx} < 256 \quad \text{and} \quad 0 \leq \text{offset} < 256.$$

The first forward helper is the extreme case: `block` < 1 , `j` < 128 , and `offset` $= j + 128$. The remaining helpers instantiate the same shape for smaller stage lengths and more blocks. The inverse helpers use the same address structure in reverse stage order. The final inverse scale loop has only the single obligation $0 \leq j < 256$.

The twiddle-table accesses are fixed by construction. The forward transform uses `jzetas[1]` through `jzetas[255]`; the inverse butterfly stages use `jzetas_inv[0]` through `jzetas_inv[254]`; and the inverse final scale reads `jzetas_inv[255]`. These are all inside the 256-entry table types.

Base multiplication has an even simpler safety shape. The public loop counter `i` ranges from 0 to 255, and the only source-level memory operations are `ap[i]`, `bp[i]`, and `rp[i]`. Since the wrapper type gives all three pointers type `ptr u32[HAETAEN]` and `HAETAEN = 256`, the safety

proof reduces to the loop-bound invariant $0 \leq i < 256$. The `const` qualifier on `ap` and `bp` also matches the intended data flow: the source polynomials are read, and only `rp` is written.

6 Constant-Time Argument

The regular constant-time checker enforces the CCS-style leakage discipline syntactically. Any expression that determines a branch, loop guard, memory address, or array index must be public. Coefficients and Montgomery products may be secret, but they must not control the trace.

For the forward and inverse NTT, all coefficient-derived values stay in arithmetic positions. A loaded coefficient may be added, subtracted, cast, shifted, or multiplied in Montgomery arithmetic, and the result may be stored back to a polynomial cell. It never flows into a branch condition, loop bound, or address expression. The loop schedule is fixed by stage constants, and the twiddle-table indices are public expressions such as `1+block`, `64+block`, or `254+block`.

For base multiplication, the trace is the public 256-iteration load/load/store pattern shown in Listing 4. The loaded values `a` and `b` are secret. The product `r` is secret. However, the branch condition is `i < 256`, and every memory address uses the same public index `i`. Thus the coefficient values determine only the words written to `rp`, not which instructions execute or which locations are touched.

The regular `ct` annotations on the exported wrappers record the security dependency of results without forcing polynomial data to be public. For `poly_ntt_jazz` and `poly_invntt_jazz`, the annotation `rp -> rp` states that the returned pointer/result has the same security dependence as the input polynomial argument. For `poly_basemul_jazz`, the annotation

$$\text{rp} * \text{ap} * \text{bp} \rightarrow \{\text{ap}, \text{bp}, \text{rp}\}$$

uses `rp`, `ap`, and `bp` as named security levels. The result label is conservative: the returned pointer is allowed to depend on all three input labels even though the source body returns the destination pointer. The essential constant-time obligation is separate from minimality of this label: the checker still requires the indices in `ap[i]`, `bp[i]`, and `rp[i]` to be public.

7 Speculative Constant-Time

The speculative checker strengthens the regular property for transient execution. The implementation uses `sct` annotations that distinguish a pointer’s security level from the security level of values reachable through the pointer:

$$\{\text{ptr} : \ell_p, \text{val} : \ell_v\}.$$

The exported wrappers accept transient pointers to secret polynomial values and return public pointers to secret

polynomial values. Intuitively, the pointer base is allowed to become public after the entry speculation boundary, while the coefficients remain secret throughout.

The source instruction `#init_msf()` is the hardening boundary. In the checker it initializes the microarchitectural speculation state; in the x86-64 lowering used here, the exported assembly begins with an `lfence`. After that point, the NTT and base-multiplication bodies use only public pointer bases and public counters for addressing. Secret values loaded from polynomial cells flow into arithmetic and stores, not into transient address computation.

This matters for the public ABI boundary. A regular constant-time proof would rule out leaks from committed branches and committed addresses. The speculative proof also prevents the reasoning from silently assuming that a public-looking pointer or bound remains public before the entry fence. In this implementation there are no secret-dependent bounds to harden inside the polynomial kernels; the only required speculation boundary in the Jasmin SCT model for this call graph is at the exported entry point. This is not a claim about all speculative execution variants, data-dependent prefetching, frequency effects, or other microarchitectural channels outside that model.

8 Compiler-Level Guarantee

The checks above are run on `jasmin/hpoly.jazz`, not on a hand-written assembly proof. This is meaningful because Jasmin is designed around a predictable compiler whose transformations preserve the source-level discipline [1, 2, 14]. The compiler avoids optimizations that are dangerous for constant-time code, such as replacing secret arithmetic by table lookups or introducing case splits on secret-dependent values. Public loop structure may be lowered into branches and indexed memory operands, but secret values must not become branch predicates or address selectors. Thus the formal claim is a composition of source-level checker results with the Jasmin compiler’s preservation theorem and timing discipline; the assembly inspection below is a sanity check that the generated code has the expected public-counter shape.

This paper does not use the newer front-end cryptographic-security preservation theorem as a premise for HAETAE. That theorem is cited as related work on the gap between source-level cryptographic proofs and compiled code; it is front-end-focused and KEM/IND-CCA-focused, whereas the present result is a source-to-assembly safety and timing argument for scalar NTT kernels.

The generated scalar assembly has the expected shape. Each exported wrapper contains an entry `lfence`. The base-multiplication body lowers `ap[i]`, `bp[i]`, and `rp[i]` to base-plus-scaled-index memory operands with the public loop counter as the scaled index, and the loop branch compares that counter with the constant 256. The NTT stage loops likewise branch on public counters and form addresses from the public base pointer, public stage constants, and public counters. Coefficients appear in arithmetic reg-

Command	Result
<code>jasminc -checksafety -o /tmp/hpoly-safe.s jasmin/hpoly.jazz</code>	No safety violation for <code>poly_ntt_jazz</code> , <code>poly_invntt_jazz</code> , and <code>poly_basemul_jazz</code> .
<code>jasmin-ct jasmin/hpoly.jazz</code>	Regular constant-time check passed.
<code>jasmin-ct -speculative jasmin/hpoly.jazz</code>	Speculative constant-time check passed; exported wrappers have transient pointer inputs and public pointer, secret value outputs.

Table 6: Compiler-side verification evidence.

isters and stored words, but not in branch predicates or address registers.

The compiler-level guarantee should therefore be read as a conditional composition: source-level safety, source-level constant-time checks, speculative entry hardening, and Jasmin’s preservation discipline together justify using the generated assembly as the checked scalar implementation. The claim is stronger than a manual style audit of the assembly because the argument is tied to the source contracts and compiler invariants, but it remains relative to the Jasmin semantics, checker model, and compiler version identified in the artifact.

9 Verification Evidence

The commands in Table 6 were run on the artifact. The safety checker reports no safety violation for all three exported routines. The regular constant-time checker exits successfully. The speculative checker exits successfully and prints the expected public-pointer/secret-value signatures for the internal kernels and exported wrappers.

For reproducibility, the reported run used `jasminc -version = Jasmin Compiler 2026.03.0` and repository revision `f0e1399`. The raw logs are included in `paper/logs/`: `jasminc-version.log`, `git-revision.log`, `jasmin-checksafety.log`, `jasmin-ct.log`, `jasmin-ct-speculative.log`, and `jasmin-compile.log`. The generated assembly from `jasminc jasmin/hpoly.jazz -o /tmp/hpoly-after.s` was byte-for-byte identical to the checked `hpoly.s`; the artifact records this through `assembly-sha256.log` and the empty `assembly-diff.log`. A submitted artifact should also identify the exact Jasmin compiler source or container image, because the safety and timing claims are relative to the checker implementation used for these commands.

Table 7 summarizes the reviewer-facing evidence that was checked against the raw logs and generated assembly. The table is not a substitute for the logs; it records which concrete artifacts support each claim so that the paper remains auditable without relying on prose alone.

A minimal reproduction run from the repository root has four steps. First, record the tool and source versions with `jasminc -version` and

Artifact evidence	What it shows
<code>jasmin-checksafety.log</code>	All three exports report No Safety Violation ; the base-multiplication report names <code>mem_rp</code> , <code>mem_ap</code> , and <code>mem_bp</code> .
<code>jasmin-ct-speculative.log</code>	Internal kernels are typed with public pointer bases and secret values; exported wrappers accept transient pointers and return public pointers to secret values.
<code>hpoly.s</code>	Each exported wrapper begins with <code>lfence</code> ; loops compare public counters with constants and use base-plus-scaled-index memory operands.
<code>assembly-sha256.log</code> , <code>assembly-diff.log</code>	The regenerated assembly and checked <code>hpoly.s</code> have identical SHA-256 hashes, and the recorded diff is empty.

Table 7: Raw evidence used to audit the checker and assembly claims.

Surface	Size	Checked role
Constants and tables	69 source lines	<code>params.jinc</code> and <code>zetas.jinc</code> fix q , n , and the twiddle layout used by the stage-specialized loops.
Arithmetic helpers	34 source lines	<code>reduce.jinc</code> supplies <code>__montgomery_reduce</code> and <code>__fqmul</code> ; the check keeps their arithmetic results away from branches and addresses.
Polynomial kernels	698 source lines	<code>poly.jinc</code> contains eight forward stages, eight inverse stages, final scaling, and base multiplication.
ABI wrappers	38 source lines	<code>hpoly.jazz</code> exports the three checked routines and attaches the constant-time/speculative constant-time boundary annotations.
Generated assembly	908 assembly lines	<code>hpoly.s</code> contains the three exported wrappers, each with an entry <code>lfence</code> , and the compiled stage loops.
Raw verification logs	211 log lines	<code>paper/logs/*.log</code> records the compiler version, source revision, checker outputs, assembly hashes, and empty assembly diff.

Table 8: Artifact size and checked surface. The table is an audit summary, not a runtime benchmark.

`git rev-parse -short HEAD`. Second, run the three source checks: `jasminc -checksafety -o /tmp/hpoly-safe.s jasmin/hpoly.jazz`, `jasmin-ct jasmin/hpoly.jazz`, and `jasmin-ct -speculative jasmin/hpoly.jazz`. Third, regenerate assembly with `jasminc jasmin/hpoly.jazz -o /tmp/hpoly-after.s`. Finally, compare `hpoly.s` with the regenerated file, for example by SHA-256 and an ordinary diff. The expected result is exactly the one recorded in `paper/logs/`: no safety violation, successful regular and speculative constant-time checks, matching assembly hashes, and an empty assembly diff.

The table also defines when the evidence must be regenerated. A change to `hpoly.jazz`, `poly.jinc`, `reduce.jinc`, `zetas.jinc`, or `params.jinc` changes the checked source surface. A change to the Jasmin compiler version changes the checker implementation. A change to `hpoly.s` changes the assembly object whose fence placement and memory-address discipline are being cited. Any of these changes

invalidates the recorded evidence unless the safety, regular constant-time, speculative constant-time, and assembly-comparison logs are regenerated. Likewise, adding raw memory operations, data-dependent branches, declassification, or new wrappers without matching constant-time/speculative constant-time signatures would require a fresh review of the leakage argument, not only a rerun of the commands.

The safety output for base multiplication is particularly informative because it names the three memory arguments:

`mem_rp`, `mem_ap`, `mem_bp`.

The successful check confirms that the typed input-pointer assumptions are sufficient to prove all reads from `ap/bp` and all writes to `rp` safe. For the NTT wrappers, the output similarly confirms safety under the single polynomial pointer argument `rp`.

The speculative constant-time output assigns the internal kernels types of the form

$$\{\text{ptr} : \text{public}, \text{val} : \text{secret}\} \rightarrow \{\text{ptr} : \text{public}, \text{val} : \text{secret}\},$$

after the exported wrappers establish the boundary from transient input pointers. For `poly_basemul_jazz`, all three input arguments have type

$\{\text{ptr} : \text{transient}, \text{val} : \text{secret}\},$

and the output has type

$\{\text{ptr} : \text{public}, \text{val} : \text{secret}\}.$

This matches the intended policy: pointer bases become public for addressing, while polynomial contents remain secret.

10 Boundaries and Trust

The result is intentionally scoped. It covers the scalar Jasmin routines in the artifact, not an AVX2 implementation and not the full HAETAE signature scheme. It proves safety and constant-time properties in the Jasmin source and compiler model; it is not a measurement claim about every physical microarchitectural effect. It assumes that callers satisfy the typed pointer preconditions of the exported routines: the input pointers must designate valid 256-word polynomial regions, and the ABI must call the generated assembly according to its signature.

The claim is also separate from functional correctness. The companion EasyCrypt development currently proves the extracted forward and inverse NTT wrappers against the abstract transform. It does not yet include a complete end-to-end functional correctness proof for `poly_basemul_jazz`; for that routine, this paper claims only safety and timing discipline. Safety and constant-time do not by themselves show that the transform is

the mathematically intended NTT; they show that the compiled implementation has defined executions and a fixed branch/address trace for all coefficient values. Conversely, a functional proof over an extracted model needs this compiler-side layer before it can be used as a high-assurance implementation claim.

The functional proof boundary is worth stating precisely because the checked Jasmin source in this paper is helperized for safety. The companion EasyCrypt chain first extracts the helperized source as `Hpoly_extract.ec`. A pRHL bridge in `CTLoopEquiv.ec` relates that helperized extraction to a proof-only direct-loop model `Hpoly_loop.ec`. The refinement proof then connects the direct-loop model to the imperative specification in `NTT_Fq.ec`, and the algebraic files `NTTFullSpec.ec` and `NTTFullAlgebra.ec` connect that imperative specification to the abstract full NTT and inverse NTT. The top-level file `NTTEndToEnd.ec` composes these steps for `poly_ntt_jazz` and `poly_invntt_jazz`. Thus the functional claim is about the same helperized source shape used by the safety and timing checks, but only for the two NTT wrappers and only under the stated representation and input-bound preconditions.

The claim is likewise separate from compiler preservation of cryptographic security in the sense of recent interaction-tree/RHL work. That line of work addresses preservation of KEM IND-CCA security through the Jasmin compiler front-end [4]. The present paper does not instantiate that theorem, does not cover the full HAETAE signature scheme, and does not derive a cryptographic security reduction for the generated assembly. It supplies the lower implementation layer needed before such higher-level claims can be read as claims about this scalar code.

11 Related Work

Jasmin introduced a language and verified compiler methodology for high-speed, high-assurance cryptographic code [1]. The Last Mile work demonstrated how Jasmin and EasyCrypt can connect optimized low-level implementations to functional-correctness and side-channel arguments [2]. Later Jasmin presentations document the workbench view: source-level reasoning, automatic safety and constant-time checkers, EasyCrypt support, and continuing work on speculative security [11, 8, 6, 14]. This paper does not extend the Jasmin theory or checker implementation. Its contribution is instead an artifact-level instantiation: the dynamic NTT schedule is expressed as fixed helpers so that the existing safety, regular constant-time, and speculative constant-time checkers can discharge the concrete HAETAE source.

Computer-aided cryptography has developed several approaches to connecting cryptographic specifications to executable code [5]. HACLS* verifies portable cryptographic implementations in an F*-based toolchain [15]; Fiat-Crypto generates and proves arithmetic code from high-level specifications [7]; and recent work on Kyber

uses Jasmin and proof tooling for implementation correctness of post-quantum arithmetic [3]. The present work is narrower: it reports the safety and timing side of one scalar HAETAE NTT artifact, rather than a general code-generation method or full cryptographic library. This narrower scope is deliberate. It isolates the compiler-side layer that a functional EasyCrypt proof must rely on before being read as a statement about the generated x86-64 implementation.

The newest compiler-security work for Jasmin attacks a different gap: it formalizes probabilistic cryptographic security with interaction trees and proves preservation for the compiler front-end, focusing on KEM IND-CCA security [4]. That result is complementary to the present artifact. It motivates precise source-to-assembly statements, but it does not remove the need to state separately what safety, constant-time, SCT, and functional-correctness evidence this HAETAE NTT artifact actually provides.

12 Conclusion

The scalar HAETAE NTT implementation was written in a form that exposes its security-relevant structure to the Jasmin compiler toolchain. Stage specialization makes polynomial and table bounds visible to the safety checker. Regular constant-time annotations express that coefficient values may be secret but cannot drive branches or addresses. Speculative annotations and `#init_msf()` establish a public entry boundary for pointer bases while preserving secret polynomial contents.

The resulting artifact passes the Jasmin safety checker, regular constant-time checker, and speculative constant-time checker for the three exported scalar routines. Together with the Jasmin compiler’s preservation discipline, these checks provide a concrete source-to-assembly guarantee: the generated scalar x86-64 code is safe under its typed pointer preconditions and has a fixed branch and memory-address schedule independent of secret polynomial coefficients. This compiler-side result complements, but does not replace, the separate EasyCrypt functional proof currently available for the forward and inverse NTT wrappers.

The paper is therefore suitable as a focused verification-artifact submission: it has a clear implementation target, a reproducible checker record, a stated leakage model, and explicit trust boundaries. It should not be positioned as a complete implementation-correctness paper for all HAETAE polynomial arithmetic until the base-multiplication functional proof, full integration theorem, and performance comparison are added. Those are extensions of the contribution, not hidden premises of the safety and timing result reported here.

References

- [1] J. B. Almeida, M. Barbosa, G. Barthe, A. Blot, B. Gregoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub. Jasmin: High-assurance and high-speed cryptography. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1807–1823. ACM, 2017.
- [2] J. B. Almeida, M. Barbosa, G. Barthe, B. Gregoire, A. Koutsos, V. Laporte, T. Oliveira, and P.-Y. Strub. The last mile: High-assurance and high-speed cryptographic implementations. In *2020 IEEE Symposium on Security and Privacy*. IEEE, 2020.
- [3] J. B. Almeida, M. Barbosa, G. Barthe, B. Gregoire, V. Laporte, J.-C. Lechenet, T. Oliveira, H. Pacheco, M. Quaresma, P. Schwabe, A. Sere, and P.-Y. Strub. Formally verifying kyber: Episode IV: Implementation correctness. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):164–193, 2023.
- [4] S. Arranz-Olmos, G. Barthe, L. Blatter, B. Gregoire, V. Laporte, and P. Torrini. The jasmin compiler preserves cryptographic security: Formalizing cryptographic security with itrees and proving compiler preservation with rhl. arXiv:2511.11292, 2025. Version 1, 14 November 2025.
- [5] M. Barbosa, G. Barthe, K. Bhargavan, B. Blanchet, C. Cremers, K. Liao, and B. Parno. Sok: Computer-aided cryptography. In *42nd IEEE Symposium on Security and Privacy*, pages 777–795. IEEE, 2021.
- [6] Engineering Cryptographic Software course. Engineering cryptographic software: High-assurance crypto and jasmin. Course slides, 2024.
- [7] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala. Simple high-level code for cryptographic arithmetic: With proofs, without compromises. In *2019 IEEE Symposium on Security and Privacy*, pages 1202–1219. IEEE, 2019.
- [8] B. Gregoire. The jasmin workbench for high-assurance and high-speed cryptography. Presentation slides, June 2023.
- [9] B. Gregoire and S. Priya. Latest improvements in the jasmin compiler and protection against spectre attacks. Presentation slides, June 2022.
- [10] HAETAE Team. HAETAE: Lattice-based digital signature algorithm. Submission to the Korean Post-Quantum Cryptography Competition, 2022. <https://kpmc.or.kr/competition.html>.
- [11] V. Laporte and the Jasmin contributors. Jasmin: a certified workbench for high-assurance and high-speed cryptography. Coq Workshop slides, July 2021.
- [12] P. L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.
- [13] J. M. Pollard. The fast Fourier transform in a finite field. *Mathematics of Computation*, 25(114):365–374, 1971.
- [14] P. Schwabe. Engineering cryptographic software: The jasmin framework. Course slides, Jan. 2026.
- [15] J. K. Zinzindohoue, K. Bhargavan, J. Protzenko, and B. Beurdouche. HACL*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1789–1806. ACM, 2017.